

Some Proposals for MAVLink 2.0

This is based on an email discussion here:

<http://lists.dronecode.org/pipermail/dronecode-tsc/2015-October/000164.html>

For detailed comments please use the dronecode-tsc mailing list.

Pain Points with MAVLink 1.0

This is a set of proposals intended to be part of the upcoming MAVLink 2.0 protocol. Note that there is considerable overlap between some of the suggestions in this email and some of our previous discussions.

To start with let's look at the major pain points and deficiencies of the current MAVLink 1.0 protocol. We will label each pain point P_n, for pain point n.

- P1) the 8 bit opcode means that we are running out of messages, plus it makes us use multiple compile-time dialects that make different MAVLink sub-projects unable to interoperate
- P2) the lack of support for message signing means that bare MAVLink over an insecure transport is very vulnerable to attack by malicious users
- P3) the fixed framing format means it is hard to evolve the protocol. We don't have any flags or room for option bytes

Those are the really key pain points, but there are additional things that have also been painful, although to a lesser degree:

- P4) the lack of variable length arrays (and more complex data structures in general)
- P5) the difficulty of adding a new field to an existing message
- P6) there is no mechanism to detect routing loops, such as a "time to live". This means that with complex topologies it is possible for the links to be flooded with continuously routed packets
- P7) Allowing the consolidation of data formats between telemetry and logging

A lot of the discussion and effort on MAVLink 2.0 up to now has concentrated on P4 and P5. These are great things to solve, but we need to put some more effort into the first 3.

We particularly need to concentrate on P1, as the full ramifications of the current opcode system may not be obvious to everyone. The 256 message limit being bad is pretty obvious, but perhaps more important is the way that it divides the dronecode community.

Dialects in MAVLink 1.0

In MAVLink 1.0 we have a common.xml plus 9 different "dialects" of MAVLink, each one associated with a MAVLink using project. The common.xml core messages use message IDs from 0 to 179 and 241 to 254. The other dialects can use message IDs from 180 to 240, which means they overlap.

When you build the C version of the MAVLink language bindings you get the common set of messages plus one of the dialects. That means if you build the "pixhawk" dialect then the resulting code will not be able to encode or decode the "ardupilotmega" dialect. That is really bad as it means our basic dronecode communication protocol provides a technical barrier to interoperability between projects.

The other language bindings handle this in different ways. For example the python bindings do build all dialects, but you have to choose which dialect to decode at runtime using an environment variable or a command line switch. There is no way to automatically detect and use the right dialect. So it is a bit better than the C bindings, but not much.

There are flow-on effects from this as well. When we add a message to one dialect (say "ardupilotmega"), that message is not available to all projects. To combat this we would like to put as many messages as possible into common.xml. The problem is we then have less message IDs available for all dialects, and to retain compatibility with existing deployed systems we end up with some duplicate messages (for example, the "RADIO" and "RADIO_STATUS" messages) which further reduces the number of available message IDs.

There is a good aspect to the current dialect system however. It allows projects to allocate IDs without having to mediate via a central master list. As long as the project adds a message within the 180 to 240 range you know that the same message ID won't be used by a different project for a different message. It still means the projects need to mediate message allocation within their contributors, but that is a lot easier than mediating between all projects.

Proposal 1 - dialects in MAVLink 2.0

To address pain point P1 I propose we should have header bits in the next MAVLink dialect which separately identify the dialect and the message ID within that dialect. All language bindings would build all dialects, so all implementations would be able to decode, encode and display all messages, but separate dialect XML files would be kept as a method for easy ID allocation at the project level.

I think there should be an 8 bit dialect ID, plus a 16 bit message ID (opcode). There would be an common enum of dialects in common.xml. Each dialect.xml file would have a dialect number definition:

<dialect>3</dialect>

the compiler would check to make sure there are no dialect conflicts. Within a dialect you can have 16 bit message IDs. The common.xml file would get dialect 0.

I think the initial message IDs should be the same ones we have now, allowing us to keep using the existing XML files, with just the addition of the dialect marker. During the transition to the new MAVLink version we would stay within the constraints of the existing 8 bit message ID so that we can keep generating both MAVLink 1.0 and the new version from the same XML files.

Proposal 2 - flag bits in header

To address pain point P3 I think we need some flexibility in the framing format of the protocol. Many (most?) widely used protocols have option bytes or flags to allow for backwards compatible evolution of the protocol.

To keep things simple I propose that we add a flags field. The purpose of these flags is to indicate a variation from the base framing format for the protocol. For example, one of the bits may indicate that the message has a 64 bit signature appended, allowing us to add signing of packets in the future.

Specifically I propose that we add two 8 bit flags fields:

uint8_t compat_flags

uint8_t incompat_flags

The difference between these two sets of flag fields is that if an implementation sets a bit set in incompat_flags that it doesn't understand then it must discard the packet. If an implementation sees a bit set in compat_flags that it doesn't understand then it may ignore the bit.

For example, if we added a flag that indicated a totally different encoding scheme then it would be good if an older implementation that didn't understand the flag would reject the packet. That bit should be in incompat_flags.

On the other hand if we added a flag that indicated "high priority packet" then it would be good if this could be ignored by implementations that don't understand the flag. That bit would be in compat_flags.

The high bit in the incompat_flags field should be reserved to mean that the header format has been changed to include another two bytes of flags (one compat, one incompat). That allows us to go beyond 16 bits of flags if we run out.

Proposal 3 - packet signing

To address P3 I think we should use one of the flags bits from proposal 2 to indicate a signature block based on a sequence number and a hash of a shared secret key. The details of a proposed signing scheme to go along with this proposal is here:

https://docs.google.com/document/d/1ETle6qQRcaNWAmpG2wz0oOpFKSF_bcTmYMQvtTG18ns/edit?usp=sharing

The signature should be optional, and the API should expose to a program using the protocol whether the particular message was signed or not. This is to allow for devices (such as the 3DR radio) which may want to inject unsigned packets such as RADIO_STATUS without it having to calculate an expensive hash.

Proposal 4 - new payload encoding

To address pain points P4 and P5 I think we should use a flag bit to indicate an alternative data encoding scheme for the payload, such as one based on Capn'proto or similar protocol encoders that we have been discussing.

The idea is to allow us to get on with solving the major pain points now, while still allowing us to move to a new encoding scheme once the details of that scheme are worked out and the implementations are ready. Otherwise I think the process of developing MAVLink 2.0 may drag on for much longer than we want it to.

I'd also like to note that it would be perfectly possible to extend the existing MAVLink 1.0 payload encoding method to have variable length arrays in a backwards compatible fashion. Lorenz and I have discussed this before but we didn't do an implementation. I think before committing to a completely new encoding we should decide if we should first implement variable length arrays in the existing payload format as I think that is the payload encoding feature that is most often painful when adding a new message.

I'll leave pain point P6 for now. I'm still thinking about whether we should allocate a TTL byte in the fixed header or use a flag bit to indicate a TTL plus other routing information.

MAVLink 0.9 and 1.0 CRC seeding

Regarding pain point P5, the difficulty of adding a new field to an existing message, I think we can use a flag bit to handle this reasonably well.

For those who don't know, the MAVLink 0.9 protocol used a conventional CRC that was dependent only on the bytes in the packet. That meant it was possible to add a new field to a

message as long as it was added at the end. Existing deployments would just not see the new field.

We moved away from that scheme as we had too many cases where the structure of a message was changed in an incompatible way. That means for example a field changing from a uint32 to a float, or a new field being added but not at the end. This was supposed to be handled by careful review of changes to the XML file, but in practice that didn't work well and a number of cases a message was changed in an incompatible manner that led to incorrect decoding.

The MAVLink 1.0 protocol addressed this by seeding the CRC with a structure hash. So if the structure of the message changed then the CRC would change. If you change the structure then older users won't be able to decode the message, but they will know they can't decode it and reject it, which is better than getting incorrect values.

Proposal 5 - packet format unlocking

For MAVLink 2.0 I think we could use a `incompat_flags` bit to indicate which scheme we are using. By default messages would use the "seeded" scheme from MAVLink 1.0, but if a `FLAG_CRC_UNLOCKED` flag bit is set in the header then no seed is used.

So it would be possible to add a new field to an existing deployed message, but only if you deliberately set the unlocked state on that message in the XML.

This helps because the types of mistakes that were made for the 0.9 protocol tended to be either because the programmer didn't know they were breaking binary compatibility, or because they thought they knew of all the deployments using the message and thought it was safe to change the message. Having an explicit unlock flag means that a programmer has to think about unlocking a message.

So for example, say we have this message:

```
<message name="RANGEFINDER" id="173">
  <description>Rangefinder reporting</description>
  <field type="float" name="distance">distance in meters</field>
  <field type="float" name="voltage">raw voltage if available</field>
</message>
```

and we want to add `distance2` and `voltage2` fields for dual rangefinder support. If implementations of this message are not yet deployed then the programmer can just add the new fields. The CRC seeding would automatically detect if the programmer has made a mistake and it is in fact deployed.

If the programmer knows that it has already been deployed then they could do this:

```
<message name="RANGEFINDER" id="173" unlocked="1">
  <description>Rangefinder reporting</description>
  <field type="float" name="distance">distance in meters</field>
  <field type="float" name="voltage">raw voltage if available</field>
  <field type="float" name="distance2">2nd distance in meters</field>
  <field type="float" name="voltage2">2nd raw voltage if available</field>
</message>
```

The unlocked flag would indicate that this packet has been encoded with an unseeded CRC. That will allow older implementation to understand the first two fields and newer implementations to see all 4 fields.

Proposal 6 - field ordering

Related to the CRC change is the field ordering of MAVLink 1.0 versus 0.9. In MAVLink 0.9 the fields were ordered in the wire format in the same order as the XML definition. In MAVLink 1.0 we made the wire order go by alignment size of the element. So all the 64 bit elements first, then the 32 bit elements, then the 16 bit elements, then the 8 bit. That guaranteed all elements were well aligned, which makes direct access fast (and possible) on all CPUs.

If we want to be able to add fields then I think we'd need to go back to the old scheme of the wire ordering being the same as the XML ordering. That means slightly less efficient pack/unpack code. I just did a test on current APM:Plane code, and found it adds about 5k of flash size to not enable MAVLINK_ALIGNED_FIELDS. The MAVLINK_ALIGNED_FIELDS define is what is used in MAVLink 1.0 to determine if field-by-field encoding is used, or if the generated code can use wire structures directly in memory.

Transitioning to MAVLink 2.0

We also need to think about how we will handle the transition to the new mavlink version. From the experience we had with the MAVLink 0.9 to MAVLink 1.0 transition a few years back I think we should assume that both MAVLink 1.0 and the new version will have to co-exist in some form for quite some time, probably at least a year.

Exactly how they will co-exist and how much pain that will cause for users and developers depends on how we do it. There are some properties of the new system we can choose that will make that easier or harder. Specifically:

- will it be possible to build an autopilot firmware that supports both protocols, with either a runtime parameter to select which is used, or auto-detection?
- if we support dual-protocol firmwares, then how much flash space will that use? (remembering that we are close to the Pixhawk limit of 1MByte now)
- will it be possible to maintain the existing C API so that our existing code will still work when using the new protocol?

- will it be possible to use the existing XML files in a way that can generate both protocols during the transition?
- will it be possible for GCS implementations to auto-detect the MAVLink version and auto-switch with no user intervention?

I think it will be much easier to get all of these properties right with a modification to mavgen instead of a completely new payload packing system.

Other possible changes

We should also think about whether we want to expand any other header fields. Should we make the sysid 16 bit for example? It is probably easiest to leave it as 8 bit and add a incompat_flags bit for 16 bit sysid if there is a need in the future.

The same goes for the payload length. Do we need packets with a payload larger than 255 bytes?

Summary

In summary I think we can address the key pain points of MAVLink 1.0 with a relatively small change. We would add an 8 bit dialect code, expand the message ID to be 16 bits and add two 8 bit flags field. That adds 4 bytes to every packet but would give us a vastly better protocol that really helps with interoperability between projects.

The new MAVLink 2.0 header would look like this:

```
uint8_t magic;           // 0xFD
uint8_t length;         // length of payload
uint8_t incompat_flags
uint8_t compat_flags
uint8_t sequence
uint8_t sysid
uint8_t compid
uint8_t dialect
uint16_t msgid
TIMESTAMP
```

Note that some incompat_flags bits may cause the header format to change.

The overall packet structure would be:

```
header
payload
crc16
[optional signature, see signing proposal]
```

Status of implementation

As of January 2016 there is a partially complete implementation of MAVLink2 available for early testers. The documentation below describes how to build and run the implementation.

Source trees

The main source tree of the development version is at:

<https://github.com/diydrones/mavlink/>

in the “mavlink2-wip” branch.

There is a port of ArduPilot to use MAVLink2 here:

<https://github.com/tridge/ardupilot>

in the “mavlink2-wip” branch.

Testing the code

Within the mavlink git tree you can test the functionality of the MAVLink2 code using the auto-generated testsuite. An example script that builds and runs the test suite is here:

https://github.com/tridge/mavlink/blob/mavlink2-wip/pymavlink/test_generator.sh

The script does the following:

- installs pymavlink as a user python module (eg \$HOME/.local on Linux)
- uses mavgen to generate both MAVLink1 and MAVLink2 C bindings
- builds testsuites for MAVLink1 and MAVLink2 for two dialects
- runs those testsuites

The test program itself is testmav.c, here:

<https://github.com/tridge/mavlink/blob/mavlink2-wip/pymavlink/generator/C/test/posix/testmav.c>

that encodes and decodes every message in the protocol. For the MAVLink2 build it tests sending every message as both MAVLink2 and MAVLink1. That exercises the code which automatically detects the wire protocol versions of each packet and decodes it as the right protocol version.

As more features of MAVLink2 are implemented this test program will be expanded to test those features.

Python Interface

The mavlink2-wip branch also includes changes to the python code generator to support MAVLink2. A very simple test of that code is here:

<https://github.com/tridge/mavlink/blob/mavlink2-wip/pymavlink/examples/mavtest.py>

that script tests both the MAVLink1 and MAVLink2 interfaces, and also serves as an example of how MAVLink1 and MAVLink2 can be mixed in one application.

A more extensive example is provided in MAVProxy:

<https://github.com/Dronecode/MAVProxy>

the master branch of MAVProxy supports MAVLink2 using the `--mav20` command line option. It depends on having a mavlink2 capable version of pymavlink installed.

Testing with ArduPilot

The most complete test of MAVLink2 currently available is provided by using the ArduPilot SITL system. If you build the mavlink2-wip branch of ArduPilot from the above link, and have the above linked versions of MAVProxy and pymavlink installed then you can do this:

```
sim_vehicle.sh -P2 --console --map
```

to get ArduPilot SITL running with MAVLink2 support.

You can also build for any hardware target by adding `-mavlink2` to the build target, for example:

```
make px4-v2-mavlink2
```

would build for a Pixhawk with mavlink2 support.

MAVLink2 features so far

The work on mavlink2 is far from complete. So far it supports the new dialect system and associated 16 bit message IDs and 8 bit dialect IDs. It supports the new header type and auto-detection of MAVLink1 versus MAVLink2 packets. It also supports application code asking for packets to be generated as either MAVLink1 or MAVLink2.

It does not yet support key MAVLink2 features such as signing and message unlocking. Adding MAVLink2 support to ArduPilot has added about 2.5kbytes to the firmware size on the Pixhawk, which is reasonably close to what was expected. As more MAVLink2 features get added this will grow, but I expect the total cost in firmware size will be under 5k.

Signing support

The prototype implementation now supports signing. To test signing support using MAVProxy and SITL you would do this:

- install pymavlink from the above branch
- install the latest master version of mavproxy
- start SITL with the `-P2` option, as above

At the mavproxy prompt you can setup a signing key on the autopilot using the “signing setup” command, like this:

```
signing setup mypassword
```

that will create a 32 byte key by using sha256 on your password and then call the `SIGNING_SETUP` MAVLink message to send the key to the autopilot.

On the next connection with MAVProxy you can use:

```
signing key mypassword
```

to locally (within mavproxy) set the key. The password should match the one you setup on the board.

To make MAVProxy reject unsigned or badly signed packets you need to set the `allow_unsigned` setting to 0.

```
set allow_unsigned 0
```

Within ArduPilot packets on the first UART are always accepted. That UART is considered secure. This corresponds with the USB port on a Pixhawk.

So when testing if signing is working in SITL you should connect on the 2nd or 3rd UART, like this:

```
mavproxy.py --master tcp:127.0.0.1:5762 --mav20
```